Influencia de los operadores y parámetros de un algoritmo genético en la solución de tres problemas de optimización

Fernando Andrés Zuluaga Botero
Bioingeniería, Unidad 3 – Optimización metaheurística
Universidad Católica de Oriente, Rionegro, Colombia
fernando.zuluaga3427@uco.net.co

***Resumen*—Se implementó en MATLAB un algoritmo genético (AG) genérico y se aplicó a tres problemas de minimización: el problema del viajero entre 13 ubicaciones del oriente antioqueño, la función Ackley modificada y un problema de política de inventarios con restricción de igualdad. Para cada problema se programaron la función objetivo, la generación de la población inicial, un operador de cruce apropiado a su representación (cruce por orden, BLX-α y cruce aritmético) y un operador de mutación; además, se implementaron tres métodos de selección comunes (elitismo, torneo y ruleta). Se realizaron cuatro estudios encadenados, con 10 ejecuciones por escenario: método de selección, número de iteraciones, estrategia de mutación y número de dimensiones de la función Ackley. La selección por elitismo obtuvo los mejores resultados en los tres casos; la ruleta, en cambio, presentó una presión selectiva insuficiente. El AG encontró el recorrido óptimo del viajero (509 min, verificado con programación dinámica) en 4 de 10 ejecuciones y alcanzó el mínimo de Ackley ($3 - e$) con errores del orden de 10^{-9} en 3 dimensiones, pero su exactitud se degradó rápidamente al aumentar las dimensiones. En el problema de inventarios, la mutación resultó decisiva: el valor esperado de pedidos no atendidos pasó de 886.8 sin mutación a 337.8 con $P_m = 1$ y una magnitud del 20 % del rango.**

***Index Terms*—Algoritmo genético, optimización metaheurística, problema del viajero, función Ackley, control de inventarios, selección, cruce, mutación.**

I. Introducción

La naturaleza ha inspirado a disciplinas muy distintas: los mecánicos han observado aves y mamíferos para diseñar vehículos más aerodinámicos, y los ingenieros han visto en los enjambres de abejas o en las colonias de hormigas modelos de organización capaces de resolver problemas complejos sin un control central. Detrás de estas adaptaciones hay millones de años de evolución, que favorece a los individuos mejor adaptados a su entorno. Este proceso inspiró a John H. Holland, cuyo trabajo culminó en 1975 con una de las obras fundacionales de los algoritmos genéticos [1]: un método que imita algunos mecanismos de la evolución para optimizar ya no organismos, sino modelos matemáticos y físicos.

Los algoritmos de optimización metaheurística se han aplicado al diseño de ejes de transmisión [2], turbinas hidráulicas [3], [4], microrredes [5], [6] y redes de sensores [7], entre otros problemas en los que el espacio de búsqueda es grande, hay múltiples variables y restricciones, o la relación entre las variables y el desempeño es compleja. Su eficacia se debe a estrategias que exploran el espacio de soluciones sin recorrerlo exhaustivamente [8].

Un algoritmo genético, sin embargo, no es un procedimiento único: su comportamiento depende de la representación de los individuos, de los operadores de selección, cruce y mutación, y de parámetros como el tamaño de la población o el número de generaciones. Además, por su naturaleza estocástica, una sola ejecución no basta para caracterizar una configuración; es necesario repetir los experimentos y analizar la calidad, la variabilidad y el costo computacional de los resultados [9].

En este trabajo se construyó un AG genérico en MATLAB y se estudió experimentalmente la influencia de sus operadores y parámetros en tres problemas: el problema del viajero (combinatorio), la función Ackley (continua, con muchos mínimos locales y dimensión variable) y un problema real de política de inventarios (continuo y con restricciones).

II. Marco teórico

II-A. Problemas de optimización

Optimizar consiste en hallar el vector $\vec{X}^*$ que minimiza (o maximiza) una función F dentro de su dominio válido Ω:

$$F(\vec{X}^*) = \min_{\vec{X} \in \Omega} F(\vec{X}), \tag{1}$$

$$\Omega = \left\{ \vec{X} \in \mathrm{dom}(f) \mid g_i(\vec{X}) \leq 0,\ h_j(\vec{X}) = 0\ \ \forall i, j \right\}, \tag{2}$$

donde f es la función objetivo o de aptitud (*fitness*), g_i son las restricciones de desigualdad y h_j las de igualdad. En los tres problemas estudiados se busca un mínimo.

II-B. Algoritmo genético

El AG trabaja con una población de soluciones candidatas (individuos). Cada individuo tiene un genotipo (su codificación: una permutación o un vector real) y un fenotipo (la solución que representa), cuya calidad se mide con la función de aptitud. En cada generación se *seleccionan* los individuos que se reproducen, se *cruzan* para crear descendientes que combinan la información de sus padres y se *mutan* algunos de ellos para introducir variaciones aleatorias que mantengan la diversidad [1], [10]. El proceso se repite durante un número determinado de generaciones.

II-C. Métodos de selección

- **Elitismo:** se ordenan los individuos por aptitud y se conserva la mejor mitad.
- **Torneo:** se forman grupos aleatorios de 4 individuos (sin repetir individuos entre grupos) y de cada grupo pasan los 2 mejores.
- **Ruleta:** cada individuo se escoge con una probabilidad proporcional a su aptitud. En minimización, esta probabilidad es inversamente proporcional al valor de la función objetivo:

$$P(\vec{X}_i) = \frac{1/f(\vec{X}_i)}{\sum_{j=1}^{n} 1/f(\vec{X}_j)}. \tag{3}$$

Estos métodos difieren en su *presión selectiva*, es decir, en qué tanto favorecen a los mejores individuos [11]. Una presión alta acelera la convergencia pero puede reducir la diversidad; una presión baja mantiene la diversidad pero hace más lenta la búsqueda.

II-D. Métricas de desempeño

La exactitud se mide con la raíz del error cuadrático medio respecto al óptimo conocido,

$$\mathrm{RMSE} = \sqrt{\frac{1}{n_p}\sum_{i=1}^{n_p}\Big(Y_i - f(\vec{X}^*)\Big)^2}, \tag{4}$$

donde Y_i es el mejor valor de la i-ésima ejecución y n_p el número de ejecuciones. La precisión se mide con la desviación estándar de los Y_i, y el costo computacional con el tiempo de ejecución.

III. Metodología

Todo el código se implementó en MATLAB R2026a (ver Anexo). Cada operador es una función independiente y un único programa, `algoritmo_genetico.m`, recibe los operadores de cada problema como *function handles*.

III-A. Funciones objetivo

III-A1. Problema del viajero: Cada ubicación se identifica con un número del 1 al 13, en el orden de la Tabla I de la guía (UCO $= 1$, Rionegro $= 2$, …, Guatapé $= 13$), y un recorrido es una lista con esos números. La función `f_viajero` suma los tiempos T_{ij} entre ubicaciones consecutivas *incluyendo el regreso al punto de partida*, de modo que el recorrido es un ciclo cerrado. Así, la lista $[1, 2, \ldots, 13]$ da $641{+}89 = 730$ min. La matriz de tiempos no es simétrica ($T_{ij} \neq T_{ji}$), por lo que el sentido del recorrido sí importa. Si la lista no es una permutación de $1, \ldots, 13$, se suma una penalización de 10 000 min, mayor que la suma de todos los tiempos de la tabla (9 534 min).

Aunque $13! \approx 6.2 \times 10^9$ recorridos hacen inviable la fuerza bruta, el óptimo exacto sí puede obtenerse con el algoritmo de programación dinámica de Held y Karp [12], de complejidad $O(n^2 2^n)$. Se implementó únicamente como referencia para medir la exactitud del AG: el recorrido óptimo tarda **509 min** (UCO → Rionegro → El Retiro → La Ceja → La Unión → El Carmen → Santuario → Cocorná → Guatapé → El Peñol → Marinilla → San Vicente → Guarne → UCO).

III-A2. Función Ackley: Se implementó la versión modificada de la guía [13]:

$$f(\vec{X}) = -a\, e^{-b\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2}} - e^{\frac{1}{d}\sum_{i=1}^{d}\cos(c\,x_i)} + a + 3, \tag{5}$$

con $a = 20$, $b = 0.2$, $c = 2\pi$ y $x_i \in [-10, 10]$. Su mínimo global está en $\vec{X} = \vec{0}$ y vale $3 - e \approx 0.281718$, lo que se verificó numéricamente para $d = 1, 2, 3, 10$ y 100.

III-A3. Política de inventarios: El individuo es $\vec{X} = [Q_1..Q_5,\ S_1..S_5,\ K_1..K_5]$ y la función objetivo es el número esperado de pedidos no atendidos por año [14], [15]:

$$Z = \sum_{i=1}^{5} \frac{D_i}{Q_i} \int_{S_i+K_i}^{\infty} \frac{x_i - S_i - K_i}{m_i\sqrt{2\pi}\sigma_i} e^{-\frac{1}{2}\left(\frac{x_i-\mu_i}{\sigma_i}\right)^2} dx_i, \tag{6}$$

con los parámetros de la Tabla II de la guía. La integral se calculó con el comando `integral` de MATLAB tras el cambio de variable $x_i = R_i + t$, con $R_i = S_i + K_i$ y $t \in [0, \infty)$. Esto permite integrar los cinco ítems en una sola llamada (opción `ArrayValued`), con un costo de 0.73 ms por evaluación. Como verificación, se comparó con la función de pérdida de la distribución normal, $\sigma_i[\phi(z) - z(1 - \Phi(z))]$ con $z = (R_i - \mu_i)/\sigma_i$: la diferencia máxima en 200 individuos fue de 1.3×10^{-7}.

Las restricciones son $0 \le S_i \le c_{1,i}$, $0 \le K_i \le c_{2,i}$ y la igualdad

$$\sum_{i=1}^{5}\left(\frac{Q_i}{2} + S_i\right) = I = 8000. \tag{7}$$

Para Q_i, cuyo límite superior en la guía es ∞, se asumió $Q_i \in [1,\ 2I] = [1,\ 16\,000]$. El límite inferior evita la división por cero y el superior corresponde al caso extremo en que todo el presupuesto de inventario se asigna a un solo artículo sin stock de seguridad: por (7), $Q_i/2$ nunca puede superar I.

III-B. Poblaciones iniciales

- **Viajero** (`pob_viajero`): n permutaciones aleatorias de $1, \ldots, 13$ generadas con `randperm`, por lo que no hay elementos repetidos.
- **Ackley** (`pob_ackley`): matriz $n \times d$ de valores uniformes en el intervalo $[-10, 10]$.
- **Inventarios** (`pob_inventario`): se generan 14 variables. S_i y K_i son uniformes en sus límites. $Q_1, \ldots, Q_4$ se toman como fracciones aleatorias (distribución de Dirichlet) del presupuesto que queda libre, $I - \sum S_i$. Finalmente, Q_5 se despeja de (7). Este procedimiento garantiza $Q_5 > 0$ sin necesidad de descartar individuos, y se verificó que todos los individuos cumplen (7) exactamente y respetan los límites.

III-C. Operadores de cruce

Se escogió un cruce adecuado a la representación de cada problema. En todos los casos, cada pareja de padres genera dos hijos.

- **Viajero, cruce por orden (OX)** [16]: se eligen dos puntos de corte; el hijo copia el segmento central de un

padre y completa las demás posiciones con las ciudades del otro padre en el orden en que aparecen, sin repetir. Los hijos siempre son permutaciones válidas y conservan el orden relativo de las ciudades, que es lo que determina el tiempo del recorrido.

- **Ackley, BLX-α** [17]: para cada gen, con $C_{\text{mín}} = \text{mín}(p_1, p_2)$, $C_{\text{máx}} = \text{máx}(p_1, p_2)$ e $I = C_{\text{máx}} - C_{\text{mín}}$, el hijo toma un valor uniforme en $[C_{\text{mín}} - \alpha I,\ C_{\text{máx}} + \alpha I]$, con $\alpha = 0.5$, y luego se recorta al dominio. Al permitir valores fuera del intervalo entre los padres, el operador también explora.
- **Inventarios, cruce aritmético** [17]: con $\lambda \sim U(0, 1)$, los hijos son $h_1 = \lambda p_1 + (1-\lambda)p_2$ y $h_2 = (1-\lambda)p_1 + \lambda p_2$. Como son combinaciones convexas de padres factibles, los hijos cumplen automáticamente la igualdad lineal (7) y los límites de caja. Por eso este cruce es especialmente apropiado para este problema.

III-D. Operadores de mutación

Cada hijo muta con probabilidad P_m (por individuo), que es un parámetro modificable.

- **Viajero:** se intercambian dos posiciones escogidas al azar (*swap*), por lo que el hijo sigue siendo una permutación.
- **Ackley:** a un gen escogido al azar se le suma $r \sim \mathcal{N}(0, \sigma_m^2)$ y el resultado se recorta a $[-10, 10]$.
- **Inventarios:** se escoge una de las 14 variables independientes y se le suma $r \sim \mathcal{N}(0, (\sigma_r \cdot \text{rango})^2)$, donde el rango es el ancho del intervalo permitido para esa variable. Así se tienen en cuenta las distintas escalas de las variables. Luego se recalcula Q_5 con (7); si queda fuera de sus límites, se descarta la mutación y se intenta de nuevo, hasta un máximo de 10 intentos.

III-E. Esquema generacional

En cada generación (Fig. 1) se selecciona la mitad de la población, se forman parejas al azar, cada pareja produce dos hijos y los hijos se mutan. La nueva población está formada por los padres seleccionados y sus hijos, de modo que su tamaño no cambia. El algoritmo guarda el mejor individuo encontrado hasta el momento, porque con la ruleta el mejor individuo puede no ser seleccionado.

III-F. Diseño experimental

Se usó una población de $n = 60$ individuos (múltiplo de 4, como exige el torneo) y 10 ejecuciones por escenario, con semillas $1, \ldots, 10$ (`rng(k)`) para que los resultados sean reproducibles. Los estudios 1 a 3 se hicieron con Ackley en $d = 3$. La configuración base fue de 100 iteraciones y $P_m = 0.2$, con $\sigma_m = 0.5$ en Ackley y $\sigma_r = 0.05$ en inventarios. Los estudios se encadenaron de la siguiente forma:

1. **Selección:** se comparan los tres métodos con 100 iteraciones y se escoge el de menor promedio (si hay empate, el de menor desviación).
2. **Iteraciones:** se prueban 10, 20, 30, 50, 100, 150 y 200 iteraciones con el método ganador. Se escoge el menor número de iteraciones cuyo promedio esté a menos del 1 % del mejor promedio.
3. **Mutación:** se varía $P_m \in \{0,\ 0.05,\ 0.1,\ 0.2,\ 0.4,\ 0.7,\ 1\}$; luego, con el mejor $P_m > 0$, se varía la magnitud de la mutación ($\sigma_m \in \{0.1, 0.5, 1, 2\}$ en Ackley y $\sigma_r \in \{0.01, 0.05, 0.1, 0.2\}$ en inventarios). $P_m = 0$ se usa como escenario de control y no se considera en la elección.
4. **Dimensiones:** se optimiza Ackley con $d = 2, 3, 10, 20, 50$ y 100, usando la configuración elegida en los estudios anteriores.

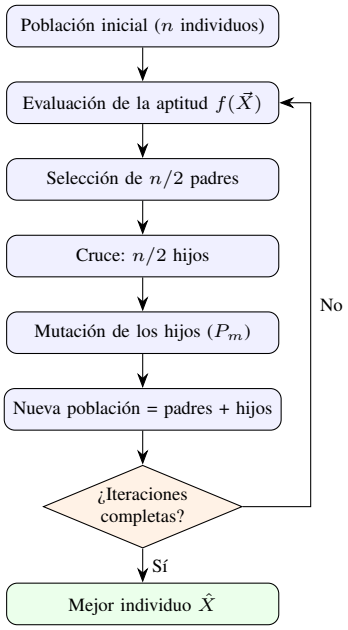


Figura 1. Diagrama de flujo del algoritmo genético implementado.

Tabla I
ESTUDIO 1 – PROBLEMA DEL VIAJERO: MÉTODO DE SELECCIÓN (100 ITERACIONES).

Método	t (ms)	$f(\hat{X})$	Prom.	Desv.	RMSE*	$\hat{X}$ (ruta)
Elitismo	36.4	509	514.1	6.19	7.78	13-12-5-10-6-1-2-7-3-9-4-8-11
Torneo	31.2	509	518.2	7.89	11.86	2-7-3-9-4-8-11-13-12-5-10-6-1
Ruleta	29.5	554	578.4	12.49	70.40	5-3-9-7-1-2-10-6-4-8-11-13-12

*RMSE respecto al óptimo exacto (509 min, Held-Karp).

En las tablas, t es el tiempo promedio por ejecución, $f(\hat{X})$ el mejor valor de las 10 ejecuciones, $\hat{X}$ el individuo que lo obtuvo, y "Prom." y "Desv." el promedio y la desviación estándar de los mejores valores de cada ejecución. En el viajero se reporta además el RMSE respecto al óptimo exacto.

IV. RESULTADOS Y ANÁLISIS

IV-A. Estudio 1: método de selección

Las Tablas I, II y III y la Fig. 2 muestran que el **elitismo** obtuvo el menor promedio en los tres problemas, seguido de cerca por el torneo. En el viajero, el elitismo encontró el óptimo exacto de 509 min en 4 de 10 ejecuciones y el torneo en 3; el promedio del elitismo (514.1 min) está apenas un 1 %

Tabla II
ESTUDIO 1 – FUNCIÓN ACKLEY ($d = 3$): MÉTODO DE SELECCIÓN (100 ITERACIONES).

Método	t (ms)	$f(\hat{X})$	Prom.	Desv.	RMSE	$\hat{X}$
Elitismo	9.65	0.281718	0.281718	1.34×10^{-9}	1.77×10^{-9}	$(-2.90\times10^{-11}, 1.22\times10^{-11}, 9.30\times10^{-12})$
Torneo	10.06	0.281718	0.281718	6.57×10^{-9}	9.41×10^{-9}	$(-9.23\times10^{-11}, 2.78\times10^{-10}, -9.28\times10^{-11})$
Ruleta	5.73	0.285186	0.290279	6.48×10^{-3}	0.011	$(1.15\times10^{-4}, -5.60\times10^{-4}, -1.37\times10^{-3})$

Tabla III
ESTUDIO 1 – INVENTARIOS: MÉTODO DE SELECCIÓN (100 ITERACIONES). VALORES MONETARIOS EN MILES DE COP.

Método	t (s)	$f(\hat{X})$	Prom.	Desv.	$\hat{Q}$	$\hat{S}$	$\hat{K}$
Elitismo	1.63	502.30	558.72	32.69	(437, 3547, 2703, 2778, 1784)	(65, 501, 510, 202, 1097)	(353, 1685, 4607, 1170, 2993)
Torneo	1.62	484.16	564.06	45.88	(414, 4221, 2432, 1567, 1952)	(40, 744, 649, 431, 845)	(413, 1177, 4985, 1483, 2946)
Ruleta	1.64	829.66	913.74	59.86	(896, 2297, 2831, 2230, 2953)	(51, 503, 473, 518, 851)	(287, 1138, 4301, 620, 2258)

Tabla IV
ESTUDIO 2 – PROBLEMA DEL VIAJERO: NÚMERO DE ITERACIONES (SELECCIÓN POR ELITISMO).

Iter.	t (ms)	$f(\hat{X})$	Prom.	Desv.	RMSE*	$\hat{X}$ (ruta)
10	8.5	551	567.9	13.69	60.32	9-3-7-2-1-8-11-5-12-13-6-10-4
20	7.5	510	539.8	19.33	35.85	8-11-13-12-5-2-1-10-6-7-3-9-4
30	10.2	509	521.5	13.69	18.02	12-5-10-6-1-2-7-3-9-4-8-11-13
50	13.9	509	517.5	8.37	11.63	13-12-5-10-6-1-2-7-3-9-4-8-11
100	26.4	509	514.1	6.19	7.78	13-12-5-10-6-1-2-7-3-9-4-8-11
150	38.2	509	514.1	6.19	7.78	13-12-5-10-6-1-2-7-3-9-4-8-11
200	56.1	509	514.1	6.19	7.78	13-12-5-10-6-1-2-7-3-9-4-8-11

*RMSE respecto al óptimo exacto (509 min, Held-Karp).

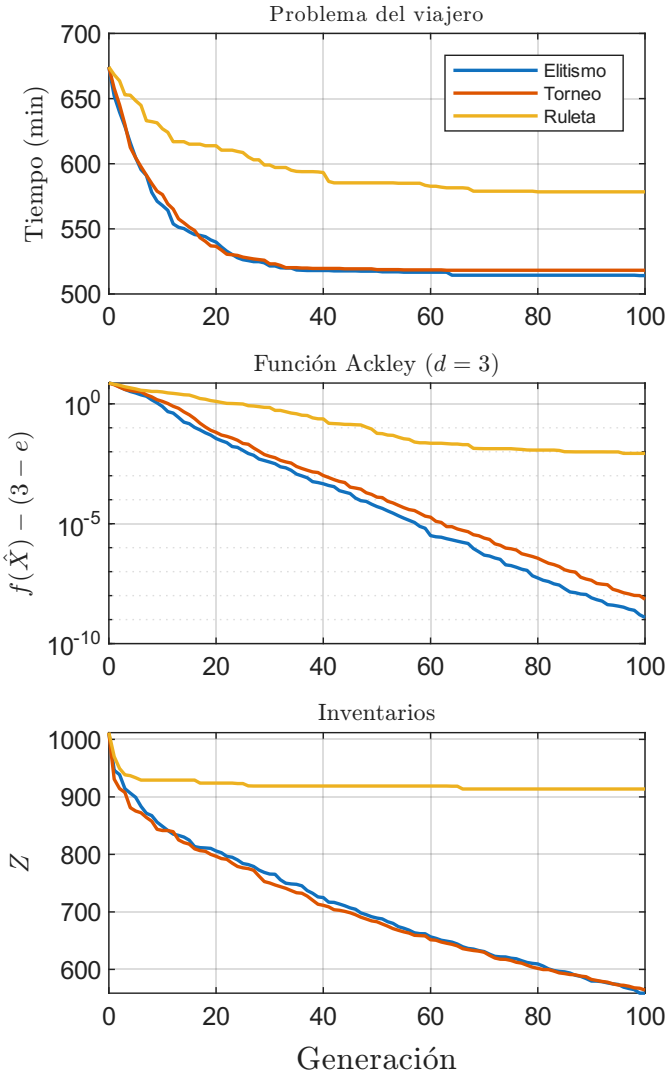


Figura 2. Convergencia promedio (10 ejecuciones) del mejor valor encontrado para cada método de selección, con 100 iteraciones. En Ackley se grafica el error respecto al óptimo en escala logarítmica.

por encima del óptimo. En Ackley ambos alcanzaron $3-e$ con errores de 10^{-9}. En inventarios la diferencia entre ambos fue pequeña frente a la dispersión: el torneo encontró el mejor valor individual (484.2) y el elitismo tuvo el mejor promedio (558.7) y la menor desviación.

La **ruleta** fue claramente la peor en los tres casos. Según (3), la probabilidad de selección depende de $1/f$, y cuando los valores de f de la población son parecidos (por ejemplo, recorridos de 520 y 560 min) sus probabilidades casi no se diferencian. La presión selectiva resulta entonces casi nula y la búsqueda se comporta de forma cercana a un muestreo aleatorio [11]. Además, al ser con reemplazo, la ruleta puede dejar por fuera al mejor individuo, cosa que no ocurre con el elitismo ni con el torneo de 4 (el mejor siempre queda entre los 2 primeros de su grupo). En inventarios esto es evidente: con ruleta, Z se estanca cerca de 910 desde las primeras generaciones (Fig. 2). La ruleta sólo fue más rápida porque ordena menos, pero la diferencia de tiempo es despreciable. Se eligió el elitismo para los estudios siguientes en los tres problemas.

IV-B. Estudio 2: número de iteraciones

Como era de esperarse, el promedio y la desviación disminuyen al aumentar las iteraciones, mientras que el tiempo crece de forma aproximadamente lineal (Tablas IV–VI). El comportamiento, sin embargo, fue diferente en cada problema:

- **Viajero:** con 30 iteraciones ya se encontró el óptimo en una ejecución. A partir de 100 iteraciones los resultados son idénticos (514.1 min en promedio), porque la población converge a un solo recorrido y la única fuente de variación que queda es la mutación. En las 6 ejecuciones que no llegaron al óptimo, el algoritmo quedó atrapado en un óptimo local. Según el criterio definido se escogieron **50 iteraciones** (517.5 min, a menos del 1 % del mejor

Tabla V
ESTUDIO 2 – FUNCIÓN ACKLEY ($d = 3$): NÚMERO DE ITERACIONES (SELECCIÓN POR ELITISMO).

Iter.	t (ms)	$f(\hat{X})$	Prom.	Desv.	RMSE	$\hat{X}$
10	2.66	0.449046	1.055537	0.456	0.886	(-0.013, -0.032, 0.039)
20	0.89	0.288317	0.317418	0.016	0.039	$(-1.75\times10^{-3}, -1.64\times10^{-3}, -1.45\times10^{-3})$
30	1.31	0.282235	0.285437	2.64×10^{-3}	4.48×10^{-3}	$(4.64\times10^{-5}, 1.56\times10^{-4}, -1.53\times10^{-4})$
50	2.57	0.281729	0.281770	6.87×10^{-5}	8.32×10^{-5}	$(1.25\times10^{-6}, 3.45\times10^{-6}, 2.83\times10^{-6})$
100	4.06	0.281718	0.281718	1.34×10^{-9}	1.77×10^{-9}	$(-2.90\times10^{-11}, 1.22\times10^{-11}, 9.30\times10^{-12})$
150	7.59	0.281718	0.281718	4.74×10^{-11}	4.74×10^{-11}	$(-1.15\times10^{-16}, -1.91\times10^{-16}, 2.19\times10^{-16})$
200	8.78	0.281718	0.281718	4.72×10^{-11}	4.72×10^{-11}	$(-4.57\times10^{-18}, -3.51\times10^{-18}, -1.46\times10^{-16})$

Tabla VI
ESTUDIO 2 – INVENTARIOS: NÚMERO DE ITERACIONES (SELECCIÓN POR ELITISMO). VALORES MONETARIOS EN MILES DE COP.

Iter.	t (s)	$f(\hat{X})$	Prom.	Desv.	$\hat{Q}$	$\hat{S}$	$\hat{K}$
10	0.22	750.40	848.82	49.11	(1100, 3202, 2390, 2519, 2169)	(59, 488, 524, 195, 1044)	(352, 1031, 3726, 851, 2574)
20	0.34	718.04	805.99	39.59	(1039, 3208, 2701, 2501, 1928)	(60, 491, 524, 200, 1036)	(349, 1334, 3719, 854, 2490)
30	0.53	677.32	765.84	37.76	(1037, 3216, 2679, 2501, 1940)	(60, 491, 524, 200, 1039)	(351, 1303, 3934, 921, 2660)
50	0.82	599.32	689.34	41.61	(440, 3369, 2699, 2775, 2060)	(66, 498, 523, 202, 1039)	(350, 1510, 3782, 925, 3000)
100	1.66	502.30	558.72	32.69	(437, 3547, 2703, 2778, 1784)	(65, 501, 510, 202, 1097)	(353, 1685, 4607, 1170, 2993)
150	4.71	431.75	470.45	23.70	(437, 3595, 2721, 2778, 1663)	(72, 501, 450, 207, 1172)	(406, 1898, 4974, 1388, 2983)
200	4.59	399.16	418.74	12.14	(328, 3576, 2721, 2772, 1573)	(74, 569, 297, 233, 1342)	(416, 1993, 4976, 1494, 2991)

Tabla VII
ESTUDIO 3 – PROBLEMA DEL VIAJERO: PROBABILIDAD DE MUTACIÓN P_m (ELITISMO, 50 ITERACIONES).

P_m	$f(\hat{X})$	Prom.	Desv.	RMSE*	$\hat{X}$ (ruta)
0.00	509	525.9	12.64	20.72	12-5-10-6-1-2-7-3-9-4-8-11-13
0.05	509	522.3	9.49	16.06	13-12-5-10-6-1-2-7-3-9-4-8-11
0.10	510	522.3	9.19	15.90	5-2-1-10-6-7-3-9-4-8-11-13-12
0.20	509	517.5	8.37	11.63	13-12-5-10-6-1-2-7-3-9-4-8-11
0.40	509	516.0	6.91	9.59	4-8-11-13-12-5-10-6-1-2-7-3-9
0.70	509	519.4	8.60	13.22	9-4-8-11-13-12-5-10-6-1-2-7-3
1.00	512	524.9	10.97	19.00	5-10-6-1-2-7-9-3-4-8-11-13-12

*RMSE respecto al óptimo exacto (509 min, Held-Karp).

promedio).

- **Ackley ($d = 3$):** con 10 iteraciones el error promedio es de 0.77. Con 50 iteraciones cae a 10^{-4} y con 100 a 10^{-9}. Se escogieron **50 iteraciones**.
- **Inventarios:** Z sigue disminuyendo hasta 200 iteraciones (de 848.8 con 10 a 418.7 con 200), lo que indica que el algoritmo no ha convergido. Es el problema más difícil: tiene 14 variables continuas con escalas muy distintas y cada evaluación requiere integrar. Se escogieron **200 iteraciones**, el máximo del estudio. Las diferencias de tiempo que no son proporcionales a las iteraciones (por ejemplo, 150 frente a 200) se deben a la variabilidad de la carga del computador.

IV-C. Estudio 3: estrategia de mutación

El efecto de P_m (Fig. 3) depende del problema:

- **Viajero:** la curva tiene forma de "U". Sin mutación el promedio es de 525.9 min: el cruce OX solo recombina lo que ya existe en la población, y el elitismo pierde diversidad rápidamente. Con $P_m = 0.4$ se obtuvo el mejor promedio (516.0 min), la menor desviación (6.9) y 4 de 10 ejecuciones en el óptimo. Con $P_m = 1$ el resultado empeora (524.9 min), porque se perturban casi todos los hijos y se destruyen buenas subrutas.
- **Ackley ($d = 3$):** con $P_m \leq 0.2$ todos los resultados son prácticamente iguales (errores promedio entre 10^{-5} y 10^{-4}), pues el BLX-α ya explora lo suficiente en 3 dimensiones. Con $P_m \geq 0.7$ la mutación aleja a los individuos del óptimo cuando la población ya está cerca de él, y con $P_m = 1$ el RMSE sube a 0.045. La magnitud σ_m tuvo poco efecto; se eligió $P_m = 0.05$ y $\sigma_m = 1$.
- **Inventarios:** la mutación es decisiva. Sin ella, $Z = 886.8$; con $P_m = 1$ y $\sigma_r = 0.05$, $Z = 355.6$; y al aumentar la magnitud a $\sigma_r = 0.2$, $Z = 337.8$ con una desviación de solo 1.9. La razón es estructural: el cruce aritmético produce combinaciones convexas de los padres, así que la población nunca puede salir del "envolvente" de la población inicial, que además se contrae en cada generación. La mutación es el único operador que puede llevar las variables a los límites. Se eligió $P_m = 1$ y $\sigma_r = 0.2$, los valores extremos de la malla, por lo que conviene explorar valores mayores en trabajos futuros.

La mejor política de inventario encontrada ($Z = 336.2$) tiene una interpretación clara. Todos los K_i quedaron en su límite superior: K_i no aparece en la restricción (7) y aumentarlo siempre reduce Z, así que el algoritmo lo lleva al máximo, como era de esperarse. El stock de seguridad se concentró en los ítems 2, 4 y 5 (en su límite), mientras que $S_3 \approx 0$ y $S_1 \approx 21$. El ítem 3 tiene el mayor valor por pedido ($m_3 = 900$) y el ítem 1 las menores ventas ($D_1 = 12\,000$), de modo que invertir en su stock de seguridad reduce poco el número de pedidos no atendidos. En cambio, Q_3 es alto: para el ítem de mayores ventas conviene hacer pedidos grandes y pocas veces.

IV-D. Estudio 4: número de dimensiones de Ackley

Con la configuración elegida (elitismo, 50 iteraciones, $P_m = 0.05$, $\sigma_m = 1$), el AG encontró el mínimo global en $d = 2$ y $d = 3$ (RMSE de 10^{-7} y 10^{-5}). A partir de $d = 10$

Tabla VIII
ESTUDIO 3 – FUNCIÓN ACKLEY ($d = 3$): PROBABILIDAD P_m Y MAGNITUD σ_m DE LA MUTACIÓN (ELITISMO, 50 ITERACIONES).

P_m / σ_m	$f(\hat{X})$	Prom.	Desv.	RMSE	$\hat{X}$
0.00 / 0.5	0.281722	0.281729	6.94×10^{-6}	1.29×10^{-5}	(1.07×10^{-6}, 1.02×10^{-6}, -2.51×10^{-7})
0.05 / 0.5	0.281722	0.281735	2.06×10^{-5}	2.57×10^{-5}	(-5.36×10^{-7}, 6.64×10^{-7}, -1.28×10^{-6})
0.10 / 0.5	0.281723	0.281996	7.65×10^{-4}	7.77×10^{-4}	(1.79×10^{-7}, -1.02×10^{-6}, -2.03×10^{-6})
0.20 / 0.5	0.281729	0.281770	6.87×10^{-5}	8.32×10^{-5}	(1.25×10^{-6}, 3.45×10^{-6}, 2.83×10^{-6})
0.40 / 0.5	0.281776	0.282203	5.57×10^{-4}	7.17×10^{-4}	(-9.59×10^{-6}, 7.00×10^{-6}, 2.22×10^{-5})
0.70 / 0.5	0.282477	0.284995	3.32×10^{-3}	4.54×10^{-3}	(-7.32×10^{-5}, 3.09×10^{-4}, 8.07×10^{-5})
1.00 / 0.5	0.294778	0.322898	0.018	0.045	(-4.97×10^{-7}, 1.66×10^{-3}, -5.17×10^{-3})
0.05 / 0.1	0.281721	0.281741	2.16×10^{-5}	3.06×10^{-5}	(-9.64×10^{-7}, -4.06×10^{-7}, 9.68×10^{-7})
0.05 / 1.0	0.281721	0.281733	1.02×10^{-5}	1.77×10^{-5}	(1.14×10^{-6}, -1.70×10^{-7}, 5.35×10^{-7})
0.05 / 2.0	0.281721	0.281791	1.37×10^{-4}	1.49×10^{-4}	(8.02×10^{-7}, -2.03×10^{-7}, -7.28×10^{-7})

Tabla IX
ESTUDIO 3 – INVENTARIOS: PROBABILIDAD P_m Y MAGNITUD RELATIVA σ_r DE LA MUTACIÓN (ELITISMO, 200 ITERACIONES). VALORES MONETARIOS EN MILES DE COP.

P_m / σ_r	$f(\hat{X})$	Prom.	Desv.	$\hat{Q}$	$\hat{S}$	$\hat{K}$
0.00 / 0.05	767.58	886.83	54.72	(1160, 3256, 2406, 2550, 2087)	(58, 481, 499, 181, 1051)	(360, 1037, 3768, 860, 2396)
0.05 / 0.05	498.47	557.72	44.42	(979, 3199, 2658, 1795, 2327)	(45, 674, 460, 492, 850)	(365, 1279, 4959, 1492, 3000)
0.10 / 0.05	452.59	487.21	23.92	(400, 3732, 2700, 1499, 2099)	(55, 675, 603, 503, 949)	(362, 1585, 4781, 1495, 2975)
0.20 / 0.05	399.16	418.74	12.14	(328, 3576, 2721, 2772, 1573)	(74, 569, 297, 233, 1342)	(416, 1993, 4976, 1494, 2991)
0.40 / 0.05	369.83	384.15	12.38	(282, 3711, 3265, 1993, 1835)	(59, 567, 18, 615, 1196)	(410, 1993, 4995, 1492, 2997)
0.70 / 0.05	349.83	364.28	8.80	(173, 3780, 2984, 1642, 1323)	(32, 772, 64, 737, 1445)	(490, 1999, 4999, 1500, 2998)
1.00 / 0.05	348.51	355.61	6.47	(203, 4143, 3056, 1451, 1403)	(22, 610, 16, 800, 1425)	(489, 1999, 5000, 1499, 2999)
1.00 / 0.01	500.89	580.91	46.21	(344, 4179, 2549, 1799, 2025)	(31, 688, 513, 415, 905)	(395, 847, 4987, 1498, 2968)
1.00 / 0.10	337.21	340.10	2.09	(94, 3089, 2955, 1504, 1325)	(26, 1195, 2, 798, 1495)	(500, 2000, 5000, 1500, 3000)
1.00 / 0.20	336.16	337.83	1.92	(118, 2936, 3219, 1423, 1262)	(21, 1200, 0, 800, 1500)	(500, 2000, 5000, 1500, 3000)

Tabla X
ESTUDIO 4 – FUNCIÓN ACKLEY: NÚMERO DE DIMENSIONES (ELITISMO, 50 ITERACIONES, $P_m = 0.05$, $\sigma_m = 1.0$).

d	t (ms)	$f(\hat{X})$	Prom.	Desv.	RMSE	$\|\hat{X}\|_\infty$
2	2.15	0.2817	0.2817	3.11×10^{-7}	3.51×10^{-7}	3.38×10^{-10}
3	2.01	0.2817	0.2817	1.02×10^{-5}	1.77×10^{-5}	1.14×10^{-6}
10	2.34	0.3323	0.4823	0.339	0.379	0.020
20	2.97	1.6879	2.0827	0.342	1.830	0.320
50	4.15	4.6696	5.4080	0.498	5.148	1.665
100	6.87	7.5101	8.0978	0.343	7.823	5.557

la exactitud se degrada rápidamente (Tabla X y Fig. 4): el RMSE pasa de 0.38 en $d = 10$ a 7.82 en $d = 100$, y la mayor coordenada del mejor individuo pasa de 0.02 a 5.6, es decir, el algoritmo termina lejos del origen y atrapado en mínimos locales. Hay tres causas principales:

1. El espacio de búsqueda crece exponencialmente con d, pero la población (60) y las iteraciones (50) se mantuvieron fijas.
2. La mutación modifica solo un gen por individuo, lo que en 100 dimensiones es una perturbación mínima.
3. La configuración se ajustó en $d = 3$, donde el problema es fácil.

El tiempo creció poco (de 2 a 7 ms) porque la evaluación de Ackley está vectorizada. Por lo tanto, tendría sentido aumentar el tamaño de la población y el número de iteraciones para problemas de alta dimensión. Los individuos completos de este estudio se incluyen como archivo anexo (`estudio4_mejores_individuos.csv`).

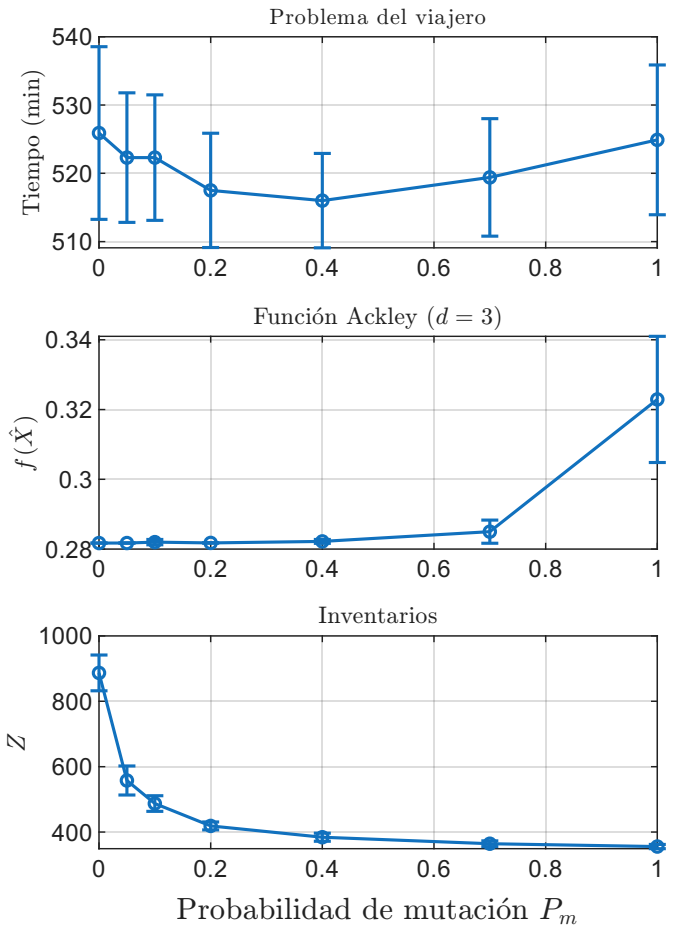


Figura 3. Promedio y desviación estándar (barras) del mejor valor en función de la probabilidad de mutación P_m.

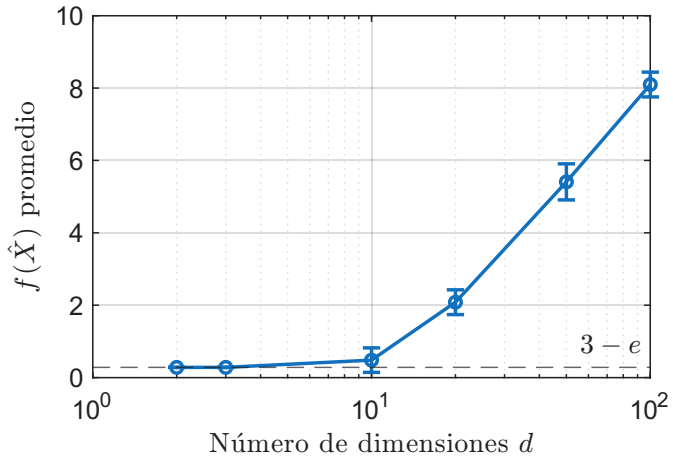


Figura 4. Promedio y desviación estándar del mejor valor de Ackley en función del número de dimensiones. La línea discontinua es el óptimo $3 - e$.

V. Conclusiones

- La selección por elitismo obtuvo los mejores promedios en los tres problemas, con el torneo muy cerca. La selección por ruleta con probabilidad inversa al fitness tuvo una presión selectiva insuficiente, porque las aptitudes de la población son muy parecidas, y fue la peor en todos los casos.
- El número adecuado de iteraciones depende del problema: 50 fueron suficientes para el viajero y para Ackley en 3 dimensiones, mientras que en inventarios el algoritmo seguía mejorando con 200. Al no conocerse el óptimo, la estabilización de la curva de convergencia y la reducción de la desviación son los criterios prácticos para decidir.
- La importancia de la mutación depende del operador de cruce. En el viajero conviene un valor intermedio ($P_m = 0.4$); en Ackley ($d = 3$) casi no tiene efecto, porque BLX-α ya explora; y en inventarios es indispensable, porque el cruce aritmético no puede salir de la envolvente convexa de la población.
- El cruce aritmético permitió manejar la restricción de igualdad del problema de inventarios sin penalizaciones, ya que conserva la factibilidad por construcción.
- El AG encontró el recorrido óptimo del viajero (509 min, verificado con Held-Karp) en el 40 % de las ejecuciones, pero su exactitud en Ackley cae con la dimensión si no se escalan la población, las iteraciones y la mutación. Esto confirma que la configuración de un AG debe ajustarse a cada problema y que su evaluación exige múltiples ejecuciones y análisis estadístico.

Referencias

[1] J. H. Holland, *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. Ann Arbor, MI: University of Michigan Press, 1975.

[2] M. A. Rodriguez-Cabal, J. D. Betancur-Gómez, L. F. Grisales-Noreña, O. D. Montoya, and D. Hincapie, "Optimal design of transmission shafts using a Vortex Search Algorithm," *Arabian Journal for Science and Engineering*, vol. 46, no. 4, pp. 3293–3300, 2021.

[3] A. J. Pérez Rodríguez, J. A. Sierra Del Río, and D. A. Hincapié-Zuluaga, "Optimización del rendimiento de una turbina Michell Banki utilizando algoritmos PSO y BHO," in *Congreso Internacional de Ingeniería con Impacto Social*, Medellín, Colombia, 2019, pp. 30–34.

[4] A. J. Perez-Rodriguez, J. Sierra-Del Rio, L. F. Grisales-Noreña, and S. Galvis, "Optimization of the efficiency of a Michell-Banki turbine through the variation of its geometrical parameters using a PSO algorithm," *WSEAS Transactions on Heat and Mass Transfer*, vol. 17, pp. 45–53, 2022.

[5] O. D. Montoya, W. Gil-González, and L. F. Grisales-Noreña, "Vortex search algorithm for optimal power flow analysis in DC resistive networks with CPLs," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 67, no. 8, pp. 1439–1443, 2020.

[6] L. F. Grisales-Noreña, D. G. Montoya, and C. A. Ramos-Paja, "Optimal sizing and location of distributed generators based on PBIL and PSO techniques," *Energies*, vol. 11, no. 4, pp. 1–27, 2018.

[7] E. H. Houssein, M. R. Saad, Y. Djenouri, G. Hu, A. A. Ali, and H. Shaban, "Metaheuristic algorithms and their applications in wireless sensor networks: Review, open issues, and challenges," *Cluster Computing*, vol. 27, no. 10, pp. 13643–13673, 2024.

[8] I. Boussaïd, J. Lepagnot, and P. Siarry, "A survey on optimization metaheuristics," *Information Sciences*, vol. 237, pp. 82–117, 2013.

[9] A. LaTorre, D. Molina, E. Osaba, J. Poyatos, J. Del Ser, and F. Herrera, "A prescription of methodological guidelines for comparing bio-inspired optimization algorithms," *Swarm and Evolutionary Computation*, vol. 67, p. 100973, 2021.

[10] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA: Addison-Wesley, 1989.

[11] D. E. Goldberg and K. Deb, "A comparative analysis of selection schemes used in genetic algorithms," in *Foundations of Genetic Algorithms*, G. J. E. Rawlins, Ed. San Mateo, CA: Morgan Kaufmann, 1991, pp. 69–93.

[12] M. Held and R. M. Karp, "A dynamic programming approach to sequencing problems," *Journal of the Society for Industrial and Applied Mathematics*, vol. 10, no. 1, pp. 196–210, 1962.
[13] D. H. Ackley, *A Connectionist Machine for Genetic Hillclimbing*. Boston, MA: Kluwer Academic Publishers, 1987.
[14] E. S. Gardner and D. G. Dannenbring, "Using optimal policy surfaces to analyze aggregate inventory tradeoffs," *Management Science*, vol. 25, no. 8, pp. 709–720, 1979.
[15] C.-B. Chen, C. T. Lin, Y.-C. Ting, and F.-K. Hsu, "Genetic algorithm for multi-item inventory control problem," *Journal of Information and Optimization Sciences*, vol. 29, no. 3, pp. 569–582, 2008.
[16] P. Larrañaga, C. M. H. Kuijpers, R. H. Murga, I. Inza, and S. Dizdarevic, "Genetic algorithms for the travelling salesman problem: A review of representations and operators," *Artificial Intelligence Review*, vol. 13, no. 2, pp. 129–170, 1999.
[17] F. Herrera, M. Lozano, and J. L. Verdegay, "Tackling real-coded genetic algorithms: Operators and tools for behavioural analysis," *Artificial Intelligence Review*, vol. 12, no. 4, pp. 265–319, 1998.

ANEXO: CÓDIGOS EN MATLAB

Los archivos se ejecutan en este orden: `pruebas.m` (verificación de los componentes) y `experimentos(1)`, `experimentos(2)`, `experimentos(3)`, `experimentos(4)`; al final, `generar_salidas` crea las tablas y figuras.

Funciones objetivo

f_viajero.m

```
function t = f_viajero(ruta)
% F_VIAJERO Funcion objetivo del problema del viajero.
% Recibe una lista con los numeros 1..13 (orden de visita) y retorna el
% tiempo total del recorrido en minutos, INCLUYENDO el regreso al punto
% de partida (ciclo cerrado). Ej.: f_viajero(1:13) = 730 (641 + 89).
% Si la lista no es una permutacion de 1..13 se penaliza sumando 10000
% (la suma de todos los tiempos de la tabla es 9534).
persistent T
if isempty(T)
   T = datos_viajero();
end
n = size(T,1);
ruta = round(ruta(:)');
penalizacion = 0;
if numel(ruta) ~= n || ~isequal(sort(ruta), 1:n)
   penalizacion = 10000;
   ruta = min(max(ruta,1),n); % se evita indexar fuera de la tabla
end
siguiente = [ruta(2:end), ruta(1)];
t = sum(T(sub2ind(size(T), ruta, siguiente))) + penalizacion;
end
```

datos_viajero.m

```
function [T, nombres] = datos_viajero()
% DATOS_VIAJERO Tabla I de la guia: tiempos de desplazamiento en minutos.
% T(i,j) es el tiempo para ir de la ubicacion i a la ubicacion j.
% La matriz NO es simetrica (T(i,j) puede ser distinto de T(j,i)).
nombres = {'UCO','Rionegro','La Ceja','El Carmen','Marinilla','Guarne', ...
   'El Retiro','Santuario','La Union','San Vicente','Cocorna', ...
   'El Penol','Guatape'};
T = [ 0 10 35 29 16 30 40 27 56 44 67 54 81
     6 0 32 31 16 32 34 28 52 44 63 56 83
    34 32 0 29 45 58 35 57 26 73 91 83 110
    29 29 28 0 32 47 57 31 49 61 67 70 99
    18 18 46 33 0 23 52 19 68 39 54 41 69
    30 30 58 46 25 0 46 36 79 39 71 62 90
    40 37 36 59 52 54 0 64 61 80 104 89 117
    32 32 57 32 18 36 65 0 82 50 41 51 82
    55 52 23 48 67 80 56 73 0 95 116 101 130
    47 47 76 62 42 37 73 53 97 0 102 81 111
    79 79 106 80 66 87 110 59 148 101 0 97 100
    59 59 86 74 44 65 90 55 107 80 93 0 29
    89 89 116 104 74 96 119 86 137 110 103 33 0];
end
```

viajero_exacto.m

```
function [t_opt, ruta_opt] = viajero_exacto()
% VIAJERO_EXACTO Optimo exacto del problema del viajero (ciclo cerrado) por
% programacion dinamica de Held-Karp. Solo se usa como referencia para
% validar el algoritmo genetico: su costo es O(n^2 * 2^n), viable para
% n = 13 pero no para problemas grandes.
T = datos_viajero();
n = size(T,1);
m = n - 1; % ciudades 2..n (la 1 es el inicio)
C = inf(2^m, m); % C(S, j): costo minimo 1 -> S -> j
P = zeros(2^m, m); % predecesor para reconstruir la ruta
for j = 1:m
   C(bitshift(1, j-1) + 1, j) = T(1, j+1);
end
for S = 1:2^m-1
   for j = 1:m
      if ~bitget(S, j) || isinf(C(S+1, j)), continue; end
      for k = 1:m
         if bitget(S, k), continue; end
         S2 = bitset(S, k);
         c = C(S+1, j) + T(j+1, k+1);
         if c < C(S2+1, k)
            C(S2+1, k) = c;
            P(S2+1, k) = j;
         end
```

```
      end
   end
end
lleno = 2^m - 1;
[t_opt, j] = min(C(lleno+1, :)' + T(2:n, 1));
ruta = zeros(1, m); S = lleno;
for pos = m:-1:1
   ruta(pos) = j + 1;
   jp = P(S+1, j);
   S = bitset(S, j, 0);
   j = jp;
end
ruta_opt = [1, ruta];
end
```

f_ackley.m

```
function y = f_ackley(X, a, b, c)
% F_ACKLEY Funcion Ackley modificada (Ecuacion 6 de la guia).
% X es un vector de d dimensiones. Se usa "+ a + 3" en lugar de "+ a + e",
% por lo que el minimo global es 3 - e (en X = 0).
if nargin < 2, a = 20; end
if nargin < 3, b = 0.2; end
if nargin < 4, c = 2*pi; end
d = numel(X);
y = -a*exp(-b*sqrt(sum(X.^2)/d)) - exp(sum(cos(c*X))/d) + a + 3;
end
```

f_inventario.m

```
function Z = f_inventario(X, metodo)
% F_INVENTARIO Numero esperado de pedidos no atendidos por falta de stock
% (Ecuacion 8 de la guia). X = [Q(1:5), S(1:5), K(1:5)].
% metodo = 'integral' (por defecto): usa el comando integral de MATLAB.
% metodo = 'cerrada': usa la funcion de perdida de la normal (solo para
% verificar el resultado numerico).
if nargin < 2, metodo = 'integral'; end
persistent p
if isempty(p)
   p = datos_inventario();
end
n = p.n;
Q = X(1:n); S = X(n+1:2*n); K = X(2*n+1:3*n);
R = S + K; % punto de reorden
switch metodo
   case 'integral'
      % Cambio de variable x = R + t, t en [0, inf): el integrando queda
      % t * f(R + t) y se integra para los 5 items a la vez (ArrayValued).
      g = @(t) t .* exp(-0.5*((R + t - p.mu)./p.sigma).^2) ./ (sqrt(2*pi)*p.sigma);
      E = integral(g, 0, Inf, 'ArrayValued', true);
   case 'cerrada'
      z = (R - p.mu)./p.sigma;
      phi = exp(-z.^2/2)/sqrt(2*pi);
      E = p.sigma .* (phi - z.*(0.5*erfc(z/sqrt(2))));
end
Z = sum(p.D ./ Q .* E ./ p.m);
end
```

datos_inventario.m

```
function p = datos_inventario()
% DATOS_INVENTARIO Parametros de la Tabla II (miles de COP).
p.D = [12000 80000 150000 50000 100000]; % ventas anuales
p.m = [20 220 900 120 180]; % valor promedio por pedido
p.mu = [300 4000 7000 2000 3500]; % media de la demanda en el tiempo de reposicion
p.sigma = [100 1200 2500 700 1100]; % desviacion estandar de esa demanda
p.Smax = [200 1200 2500 800 1500]; % c1: tope del stock de seguridad
p.Kmax = [500 2000 5000 1500 3000]; % c2: tope de la demanda definida por la empresa
p.I = 8000; % inversion promedio en inventario (Ec. 9)
% Limites asumidos para Q (la tabla dice (0, inf)):
% - Qmin = 1 (mil COP) para evitar la division por cero en D/Q.
% - Qmax = 2*I: aun si todo el presupuesto se destinara a un solo item y
% su stock de seguridad fuera cero, Q/2 no podria superar I.
p.Qmin = 1;
p.Qmax = 2*p.I;
p.n = numel(p.D);
end
```

Poblaciones iniciales

pob_viajero.m

```
function pob = pob_viajero(n, m)
% POB_VIAJERO Genera n rutas aleatorias. Cada fila es una permutacion de
% los numeros 1..m (m = 13 por defecto), sin elementos repetidos.
if nargin < 2, m = 13; end
pob = zeros(n, m);
for i = 1:n
   pob(i,:) = randperm(m);
end
end
```

pob_ackley.m

```
function pob = pob_ackley(n, d, lim)
% POB_ACKLEY Genera una matriz n x d con valores uniformes en el intervalo
% lim = [minimo, maximo]. Cada fila es un individuo.
if nargin < 3, lim = [-10 10]; end
pob = lim(1) + (lim(2) - lim(1))*rand(n, d);
end
```

pob_inventario.m

```
function pob = pob_inventario(n)
% POB_INVENTARIO Genera n individuos X = [Q(1:5), S(1:5), K(1:5)].
% Se generan 14 variables (S1..S5, K1..K5, Q1..Q4) dentro de los limites de
% la Tabla II y Q5 se calcula con la restriccion de igualdad (Ecuacion 9):
% sum(Q/2 + S) = I => Q5 = 2*(I - sum(S) - sum(Q1..Q4)/2)
% Para que Q5 siempre resulte positivo, Q1..Q4 se generan como fracciones
% aleatorias del presupuesto que queda libre despues de fijar S.
p = datos_inventario();
k = p.n;
pob = zeros(n, 3*k);
for i = 1:n
   S = p.Smax .* rand(1, k);
   K = p.Kmax .* rand(1, k);
   libre = p.I - sum(S); % presupuesto para sum(Q/2)
   w = -log(rand(1, k)); w = w/sum(w); % fracciones aleatorias (Dirichlet(1))
   Q = zeros(1, k);
   Q(1:k-1) = p.Qmin + 2*(libre - k*p.Qmin/2)*w(1:k-1);
   Q(k) = 2*(p.I - sum(S) - sum(Q(1:k-1))/2); % variable dependiente (Ec. 9)
   pob(i,:) = [Q, S, K];
end
end
```

Selección

sel_elitismo.m

```
function ganadores = sel_elitismo(fitness)
% SEL_ELITISMO Seleccion por elitismo (minimizacion).
% Ordena los individuos de menor a mayor fitness y retorna los indices de
% la mitad con mejor aptitud.
n = numel(fitness);
[~, orden] = sort(fitness(:), 'ascend');
ganadores = orden(1:floor(n/2));
end
```

sel_torneo.m

```
function ganadores = sel_torneo(fitness, tam_grupo, n_ganan)
% SEL_TORNEO Seleccion por torneo (minimizacion).
% Se forman grupos aleatorios de tam_grupo individuos (4 por defecto) sin
% repetir individuos entre grupos, y de cada grupo pasan los n_ganan
% mejores (2 por defecto). Se repite hasta tener la mitad de la poblacion.
if nargin < 2, tam_grupo = 4; end
if nargin < 3, n_ganan = 2; end
n = numel(fitness);
orden = randperm(n);
n_grupos = floor(n/tam_grupo);
ganadores = zeros(n_grupos*n_ganan, 1);
for g = 1:n_grupos
   grupo = orden((g-1)*tam_grupo + (1:tam_grupo));
   [~, pos] = sort(fitness(grupo), 'ascend');
   ganadores((g-1)*n_ganan + (1:n_ganan)) = grupo(pos(1:n_ganan));
end
end
```

sel_ruleta.m

```
function ganadores = sel_ruleta(fitness)
% SEL_RULETA Seleccion por ruleta para minimizacion (Ecuacion 12).
% La probabilidad de escoger al individuo i es inversamente proporcional a
% su fitness: P_i = (1/f_i) / sum(1/f_j). Se gira la ruleta n/2 veces
% (con reemplazo, un individuo puede salir escogido mas de una vez).
n = numel(fitness);
inv = 1 ./ max(fitness(:), eps); % se evita dividir por cero
P = inv / sum(inv);
acumulada = cumsum(P);
acumulada(end) = 1;
giros = rand(floor(n/2), 1);
ganadores = zeros(floor(n/2), 1);
for i = 1:numel(giros)
   ganadores(i) = find(giros(i) <= acumulada, 1, 'first');
end
end
```

Cruce

cruce_ox.m

```
function hijos = cruce_ox(padres)
% CRUCE_OX Cruce por orden (Order Crossover, OX) para permutaciones
% [Larranaga et al., 1999]. Se aparean las filas 1-2, 3-4, ...; cada pareja
% genera 2 hijos. Para cada hijo:
% 1. Se escogen 2 puntos de corte al azar.
% 2. El hijo copia el segmento entre los cortes de uno de los padres.
% 3. Las posiciones restantes se llenan, empezando despues del segundo
% corte, con las ciudades del otro padre en el orden en que aparecen
% en el (desde su segundo corte), omitiendo las ya copiadas.
% Asi los hijos siempre son permutaciones validas.
[np, m] = size(padres);
hijos = zeros(np, m);
for k = 1:2:np-1
   p1 = padres(k,:); p2 = padres(k+1,:);
   cortes = sort(randperm(m, 2));
   hijos(k,:) = ox_un_hijo(p1, p2, cortes(1), cortes(2));
   hijos(k+1,:) = ox_un_hijo(p2, p1, cortes(1), cortes(2));
end
end

function h = ox_un_hijo(pa, pb, c1, c2)
m = numel(pa);
h = zeros(1, m);
h(c1:c2) = pa(c1:c2);
orden_pb = pb([c2+1:m, 1:c2]); % recorrido de pb desde c2+1
restantes = orden_pb(~ismember(orden_pb, pa(c1:c2)));
posiciones = [c2+1:m, 1:c1-1]; % huecos del hijo desde c2+1
h(posiciones) = restantes;
end
```

cruce_blx.m

```
function hijos = cruce_blx(padres, alfa, lim)
% CRUCE_BLX Cruce BLX-alfa para codificacion real [Herrera, Lozano y
% Verdegay, 1998]. Para cada gen, con Cmin = min(p1,p2), Cmax = max(p1,p2) e
% I = Cmax - Cmin, el hijo toma un valor uniforme en
% [Cmin - alfa*I, Cmax + alfa*I]. Luego se recorta al dominio lim.
% Se aparean las filas 1-2, 3-4, ...; cada pareja genera 2 hijos.
if nargin < 2, alfa = 0.5; end
if nargin < 3, lim = [-10 10]; end
[np, d] = size(padres);
hijos = zeros(np, d);
for k = 1:2:np-1
   p1 = padres(k,:); p2 = padres(k+1,:);
   cmin = min(p1, p2); cmax = max(p1, p2); I = cmax - cmin;
   inf_ = cmin - alfa*I; sup_ = cmax + alfa*I;
   hijos(k,:) = inf_ + (sup_ - inf_).*rand(1, d);
   hijos(k+1,:) = inf_ + (sup_ - inf_).*rand(1, d);
end
hijos = min(max(hijos, lim(1)), lim(2));
end
```

cruce_aritmetico.m

```
function hijos = cruce_aritmetico(padres)
% CRUCE_ARITMETICO Cruce aritmetico completo [Herrera et al., 1998].
% Para cada pareja (p1, p2) se sortea lambda ~ U(0,1) y:
% h1 = lambda*p1 + (1 - lambda)*p2
```

```
% h2 = (1 - lambda)*p1 + lambda*p2
% Como los hijos son combinaciones convexas de los padres, conservan
% automaticamente la restriccion lineal de igualdad sum(Q/2 + S) = I y
% los limites de caja de todas las variables (Tabla II).
[np, d] = size(padres);
hijos = zeros(np, d);
for k = 1:2:np-1
   p1 = padres(k,:); p2 = padres(k+1,:);
   lambda = rand;
   hijos(k,:) = lambda*p1 + (1 - lambda)*p2;
   hijos(k+1,:) = (1 - lambda)*p1 + lambda*p2;
end
end
```

Mutación

mut_viajero.m

```
function hijos = mut_viajero(hijos, Pm)
% MUT_VIAJERO Mutacion por intercambio (swap). Cada individuo muta con
% probabilidad Pm: se escogen 2 posiciones distintas al azar y se
% intercambian sus valores, de modo que sigue siendo una permutacion.
[n, m] = size(hijos);
for i = 1:n
   if rand < Pm
      pos = randperm(m, 2);
      hijos(i, pos) = hijos(i, fliplr(pos));
   end
end
end
```

mut_ackley.m

```
function hijos = mut_ackley(hijos, Pm, sigma_m, lim)
% MUT_ACKLEY Mutacion gaussiana (Ecuaciones 13 y 14). Cada individuo muta
% con probabilidad Pm: se escoge un gen x_i al azar y se hace
% x_i' = x_i + r, con r ~ N(0, sigma_m^2). El resultado se recorta a lim.
if nargin < 4, lim = [-10 10]; end
[n, d] = size(hijos);
for i = 1:n
   if rand < Pm
      j = randi(d);
      hijos(i,j) = min(max(hijos(i,j) + sigma_m*randn, lim(1)), lim(2));
   end
end
end
```

mut_inventario.m

```
function hijos = mut_inventario(hijos, Pm, sigma_rel)
% MUT_INVENTARIO Mutacion gaussiana para el problema de inventarios.
% Cada individuo muta con probabilidad Pm: se escoge al azar una de las 14
% variables independientes (Q1..Q4, S1..S5, K1..K5) y se le suma
% r ~ N(0, (sigma_rel*rango)^2), donde rango es el ancho del intervalo
% permitido para esa variable (las variables tienen escalas distintas).
% El valor se recorta a sus limites y luego se recalcula Q5 con la
% Ecuacion 9. Si Q5 queda fuera de [Qmin, Qmax] se descarta la mutacion y
% se intenta de nuevo (maximo 10 intentos; si no, el individuo no cambia).
p = datos_inventario();
k = p.n;
lo = [p.Qmin*ones(1,k), zeros(1,k), zeros(1,k)];
hi = [p.Qmax*ones(1,k), p.Smax, p.Kmax];
independientes = [1:k-1, k+1:3*k];  % todas menos Q5
for i = 1:size(hijos, 1)
   if rand < Pm
      for intento = 1:10
         x = hijos(i,:);
         j = independientes(randi(numel(independientes)));
         x(j) = min(max(x(j) + sigma_rel*(hi(j) - lo(j))*randn, lo(j)), hi(j));
         x(k) = 2*(p.I - sum(x(k+1:2*k)) - sum(x(1:k-1))/2);  % Q5 (Ec. 9)
         if x(k) >= p.Qmin && x(k) <= p.Qmax
            hijos(i,:) = x;
            break
         end
      end
   end
end
end
```

definir_problema.m

```
function prob = definir_problema(nombre, d)
% DEFINIR_PROBLEMA Agrupa en una estructura las funciones de cada caso:
% prob.fitness(x) -> aptitud de un individuo (fila)
% prob.poblacion(n) -> poblacion inicial de n individuos
% prob.cruce(padres) -> hijos (mismo numero de filas que padres)
% prob.mutacion(h, Pm, sig) -> hijos mutados
% nombre: 'viajero', 'ackley' o 'inventario'. d: dimensiones (Ackley).
if nargin < 2, d = 3; end
lim = [-10 10];
switch lower(nombre)
   case 'viajero'
      prob.fitness = @f_viajero;
      prob.poblacion = @(n) pob_viajero(n, 13);
      prob.cruce = @cruce_ox;
      prob.mutacion = @(h, Pm, sig) mut_viajero(h, Pm);
   case 'ackley'
      prob.fitness = @f_ackley;
      prob.poblacion = @(n) pob_ackley(n, d, lim);
      prob.cruce = @(p) cruce_blx(p, 0.5, lim);
      prob.mutacion = @(h, Pm, sig) mut_ackley(h, Pm, sig, lim);
      prob.optimo = 3 - exp(1);
   case 'inventario'
      prob.fitness = @f_inventario;
      prob.poblacion = @pob_inventario;
      prob.cruce = @cruce_aritmetico;
      prob.mutacion = @mut_inventario;
   otherwise
      error('Problema desconocido: %s', nombre);
end
prob.nombre = lower(nombre);
end
```

algoritmo_genetico.m

```
function res = algoritmo_genetico(prob, op)
% ALGORITMO_GENETICO Algoritmo genetico generico de minimizacion.
% prob: estructura creada con definir_problema.
% op: opciones
% op.n tamano de la poblacion (multiplo de 4)
% op.iter numero de generaciones (iteraciones)
% op.seleccion handle: @sel_elitismo, @sel_torneo o @sel_ruleta
% op.Pm probabilidad de mutacion por individuo
% op.sigma magnitud de la mutacion (sigma_m en Ackley, fraccion del
% rango en inventarios; se ignora en el viajero)
% res: mejor valor encontrado, mejor individuo, curva de convergencia
% (mejor valor hasta cada generacion) y tiempo de ejecucion.
tic;
% 1. Poblacion inicial
pob = prob.poblacion(op.n);
fit = evaluar(prob, pob);
[mejor_f, im] = min(fit);
mejor_x = pob(im,:);
convergencia = zeros(op.iter + 1, 1);
convergencia(1) = mejor_f;
for g = 1:op.iter
   % 2. Seleccion: la mitad de la poblacion pasa y se aparea
   ganadores = op.seleccion(fit);
   ganadores = ganadores(randperm(numel(ganadores))); % parejas al azar
   padres = pob(ganadores,:);
   % 3. Cruce: cada pareja genera 2 hijos (la otra mitad de la poblacion)
   hijos = prob.cruce(padres);
   % 4. Mutacion de los hijos
   hijos = prob.mutacion(hijos, op.Pm, op.sigma);
   % 5. Nueva generacion = padres seleccionados + hijos
   fit_hijos = evaluar(prob, hijos);
   pob = [padres; hijos];
   fit = [fit(ganadores); fit_hijos];
   % Se guarda el mejor individuo encontrado hasta el momento
   [f_gen, im] = min(fit);
   if f_gen < mejor_f
      mejor_f = f_gen;
      mejor_x = pob(im,:);
   end
   convergencia(g + 1) = mejor_f;
end
res.mejor_f = mejor_f;
res.mejor_x = mejor_x;
res.convergencia = convergencia;
```

```
res.tiempo = toc;
end

function fit = evaluar(prob, pob)
fit = zeros(size(pob, 1), 1);
for i = 1:size(pob, 1)
   fit(i) = prob.fitness(pob(i,:));
end
end
```

correr_escenario.m

```
function r = correr_escenario(prob, op, n_pruebas, optimo)
% CORRER_ESCENARIO Ejecuta el algoritmo genetico n_pruebas veces con la
% misma configuracion (semillas 1..n_pruebas, reproducible) y resume:
% tiempo promedio, mejor valor, individuo del mejor valor, promedio,
% desviacion estandar y RMSE respecto al optimo (si se conoce).
if nargin < 4, optimo = NaN; end
Y = zeros(n_pruebas, 1);
t = zeros(n_pruebas, 1);
X = cell(n_pruebas, 1);
conv = zeros(op.iter + 1, n_pruebas);
for k = 1:n_pruebas
   rng(k);
   res = algoritmo_genetico(prob, op);
   Y(k) = res.mejor_f; t(k) = res.tiempo; X{k} = res.mejor_x;
   conv(:, k) = res.convergencia;
end
[r.mejor_f, ib] = min(Y);
r.mejor_x = X{ib};
r.promedio = mean(Y);
r.desv = std(Y);
r.rmse = sqrt(mean((Y - optimo).^2));
r.tiempo = mean(t);
r.Y = Y; r.X = X; r.conv = conv; r.op = op;
end
```

experimentos.m

```
function experimentos(etapa)
% EXPERIMENTOS Estudios del informe (seccion 3.5 de la guia). Cada etapa
% guarda sus resultados en ../resultados y usa las decisiones de la etapa
% anterior, por eso deben ejecutarse en orden: 1, 2, 3 y 4.
% 1: metodo de seleccion (100 iteraciones)
% 2: numero de iteraciones (10, 20, 30, 50, 100, 150, 200)
% 3: estrategia de mutacion (Pm y magnitud de la mutacion)
% 4: numero de dimensiones de Ackley (2, 3, 10, 20, 50, 100)
carpeta = fullfile('..', 'resultados');
if ~exist(carpeta, 'dir'), mkdir(carpeta); end
N_PRUEBAS = 10; % pruebas por escenario (la guia pide >= 5)
N_POB = 60; % tamano de poblacion (multiplo de 4)
D_ACKLEY = 3; % dimensiones de Ackley en los estudios 1-3
problemas = {'viajero', 'ackley', 'inventario'};
% Configuracion base (antes de los estudios)
base.viajero = struct('n', N_POB, 'iter', 100, 'Pm', 0.2, 'sigma', 0);
base.ackley = struct('n', N_POB, 'iter', 100, 'Pm', 0.2, 'sigma', 0.5);
base.inventario = struct('n', N_POB, 'iter', 100, 'Pm', 0.2, 'sigma', 0.05);
optimos.viajero = viajero_exacto(); % referencia (Held-Karp)
optimos.ackley = 3 - exp(1);
optimos.inventario = NaN; % desconocido
selecciones = {@sel_elitismo, @sel_torneo, @sel_ruleta};
nombres_sel = {'Elitismo', 'Torneo', 'Ruleta'};

switch etapa
   case 1
      for ip = 1:3
         nom = problemas{ip};
         prob = definir_problema(nom, D_ACKLEY);
         for is = 1:3
            op = base.(nom); op.seleccion = selecciones{is};
            E1.(nom)(is) = correr_escenario(prob, op, N_PRUEBAS, optimos.(nom));
            imprimir(nom, nombres_sel{is}, E1.(nom)(is));
         end
         % Criterio: menor promedio; empate -> menor desviacion
         [~, orden] = sortrows([[E1.(nom).promedio]', [E1.(nom).desv]']);
         decision.(nom).seleccion = selecciones{orden(1)};
         decision.(nom).nombre_sel = nombres_sel{orden(1)};
         fprintf('==> %s: mejor seleccion = %s\n', nom, nombres_sel{orden(1)});
      end
      save(fullfile(carpeta, 'E1.mat'), 'E1', 'decision', 'nombres_sel', 'optimos', 'base', 'N_PRUEBAS', 'N_POB', '
          D_ACKLEY');

```

```
   case 2
      load(fullfile(carpeta, 'E1.mat'), 'decision', 'optimos', 'base', 'N_PRUEBAS', 'D_ACKLEY');
      iteraciones = [10 20 30 50 100 150 200];
      for ip = 1:3
         nom = problemas{ip};
         prob = definir_problema(nom, D_ACKLEY);
         for ii = 1:numel(iteraciones)
            op = base.(nom); op.seleccion = decision.(nom).seleccion;
            op.iter = iteraciones(ii);
            E2.(nom)(ii) = correr_escenario(prob, op, N_PRUEBAS, optimos.(nom));
            imprimir(nom, sprintf(' %d iter', iteraciones(ii)), E2.(nom)(ii));
         end
         % Criterio: el menor numero de iteraciones cuyo promedio este a
         % menos del 1 % del mejor promedio obtenido
         prom = [E2.(nom).promedio];
         ok = find(prom <= min(prom) + 0.01*abs(min(prom)), 1, 'first');
         decision.(nom).iter = iteraciones(ok);
         fprintf('==> %s: iteraciones escogidas = %d\n', nom, iteraciones(ok));
      end
      save(fullfile(carpeta, 'E2.mat'), 'E2', 'decision', 'iteraciones');

   case 3
      load(fullfile(carpeta, 'E1.mat'), 'optimos', 'base', 'N_PRUEBAS', 'D_ACKLEY');
      load(fullfile(carpeta, 'E2.mat'), 'decision');
      valores_Pm = [0 0.05 0.1 0.2 0.4 0.7 1.0];
      valores_sigma.viajero = 0;
      valores_sigma.ackley = [0.1 0.5 1 2];
      valores_sigma.inventario = [0.01 0.05 0.1 0.2];
      for ip = 1:3
         nom = problemas{ip};
         prob = definir_problema(nom, D_ACKLEY);
         % Parte A: se varia Pm con la magnitud base
         for k = 1:numel(valores_Pm)
            op = base.(nom); op.seleccion = decision.(nom).seleccion;
            op.iter = decision.(nom).iter; op.Pm = valores_Pm(k);
            E3.(nom)(k) = correr_escenario(prob, op, N_PRUEBAS, optimos.(nom));
            imprimir(nom, sprintf('Pm=%.2f sig=%.2f', op.Pm, op.sigma), E3.(nom)(k));
         end
         % Pm = 0 es el escenario de control (sin mutacion): no se escoge
         candidatos = find(valores_Pm > 0);
         [~, orden] = sortrows([[E3.(nom)(candidatos).promedio]', [E3.(nom)(candidatos).desv]']);
         mejor_Pm = E3.(nom)(candidatos(orden(1))).op.Pm;
         % Parte B: con el mejor Pm se varia la magnitud de la mutacion
         if numel(valores_sigma.(nom)) > 1
            for s = valores_sigma.(nom)
               if s == base.(nom).sigma, continue; end
               op = base.(nom); op.seleccion = decision.(nom).seleccion;
               op.iter = decision.(nom).iter; op.Pm = mejor_Pm; op.sigma = s;
               E3.(nom)(end+1) = correr_escenario(prob, op, N_PRUEBAS, optimos.(nom));
               imprimir(nom, sprintf('Pm=%.2f sig=%.2f', op.Pm, op.sigma), E3.(nom)(end));
            end
         end
         candidatos = [2:numel(valores_Pm), numel(valores_Pm)+1:numel(E3.(nom))];
         [~, orden] = sortrows([[E3.(nom)(candidatos).promedio]', [E3.(nom)(candidatos).desv]']);
         decision.(nom).Pm = E3.(nom)(candidatos(orden(1))).op.Pm;
         decision.(nom).sigma = E3.(nom)(candidatos(orden(1))).op.sigma;
         fprintf('==> %s: Pm = %.2f, sigma = %.2f\n', nom, decision.(nom).Pm, decision.(nom).sigma);
      end
      save(fullfile(carpeta, 'E3.mat'), 'E3', 'decision', 'valores_Pm', 'valores_sigma');

   case 4
      load(fullfile(carpeta, 'E1.mat'), 'optimos', 'base', 'N_PRUEBAS');
      load(fullfile(carpeta, 'E3.mat'), 'decision');
      dimensiones = [2 3 10 20 50 100];
      for id = 1:numel(dimensiones)
         prob = definir_problema('ackley', dimensiones(id));
         op = base.ackley; op.seleccion = decision.ackley.seleccion;
         op.iter = decision.ackley.iter; op.Pm = decision.ackley.Pm;
         op.sigma = decision.ackley.sigma;
         E4(id) = correr_escenario(prob, op, N_PRUEBAS, optimos.ackley); %#ok<AGROW>
         imprimir('ackley', sprintf('d=%d', dimensiones(id)), E4(id));
      end
      save(fullfile(carpeta, 'E4.mat'), 'E4', 'decision', 'dimensiones');
end
end

function imprimir(nom, etiqueta, r)
fprintf(' %-10s %-20s t=%7.3fs mejor=%10.4f prom=%10.4f desv=%9.4f rmse=%9.4f\n', ...
   nom, etiqueta, r.tiempo, r.mejor_f, r.promedio, r.desv, r.rmse);
end
```

```
% PRUEBAS Verificacion de cada componente antes de los experimentos.
clc; rng(1);
fprintf('--- Funcion objetivo: viajero ---\n');
[T, ~] = datos_viajero();
fprintf('Suma de la tabla = %d (guia: 9534)\n', sum(T(:)));
fprintf('f(1:13) ciclo cerrado = %d (guia: 641 + regreso = 730)\n', f_viajero(1:13));
fprintf('Lista invalida [1 1 3 ... 13] = %d (debe ser >= 10000)\n', f_viajero([1 1 3:13]));

fprintf('--- Funcion objetivo: Ackley ---\n');
for d = [1 2 3 10 100]
   fprintf('d = %3d: f(0) = %.6f (3 - e = %.6f)\n', d, f_ackley(zeros(1,d)), 3-exp(1));
end

fprintf('--- Funcion objetivo: inventario ---\n');
p = datos_inventario();
X = pob_inventario(200);
err = zeros(200,1);
for i = 1:200
   err(i) = abs(f_inventario(X(i,:),'integral') - f_inventario(X(i,:),'cerrada'));
end
fprintf('Max. diferencia integral vs forma cerrada (200 ind.): %.2e\n', max(err));
tic; for i = 1:200, f_inventario(X(i,:)); end
fprintf('Tiempo por evaluacion con integral: %.3f ms\n', 1000*toc/200);

fprintf('--- Poblaciones ---\n');
P = pob_viajero(100);
fprintf('Viajero: todas son permutaciones = %d\n', all(arrayfun(@(i) isequal(sort(P(i,:)),1:13), 1:100)));
A = pob_ackley(100, 5, [-10 10]);
fprintf('Ackley: tamano %dx%d, rango [%.2f, %.2f]\n', size(A), min(A(:)), max(A(:)));
k = p.n; Q = X(:,1:k); S = X(:,k+1:2*k); K = X(:,2*k+1:end);
fprintf('Inventario: max |sum(Q/2+S) - I| = %.2e\n', max(abs(sum(Q/2,2) + sum(S,2) - p.I)));
fprintf('Inventario: limites OK = %d\n', all(Q(:) >= p.Qmin & Q(:) <= p.Qmax) && ...
   all(all(S >= 0 & S <= p.Smax)) && all(all(K >= 0 & K <= p.Kmax)));

fprintf('--- Seleccion ---\n');
f = rand(40,1);
fprintf('Elitismo: %d ganadores, son los mejores = %d\n', numel(sel_elitismo(f)), ...
   isequal(sort(sel_elitismo(f)), sort(find(f <= median(f)))));
g = sel_torneo(f);
fprintf('Torneo: %d ganadores, sin repetidos = %d\n', numel(g), numel(unique(g)) == numel(g));
cuenta = zeros(4,1);
for r = 1:20000, w = sel_ruleta([1;2;4;8]); cuenta = cuenta + accumarray(w,1,[4 1]); end
fprintf('Ruleta con f = [1 2 4 8]: frecuencia = [%s], teorica = [%s]\n', ...
   num2str(cuenta'/sum(cuenta),3), num2str((1./[1 2 4 8])/sum(1./[1 2 4 8]),3));

fprintf('--- Cruce ---\n');
H = cruce_ox(pob_viajero(40));
fprintf('OX: todos los hijos son permutaciones = %d\n', all(arrayfun(@(i) isequal(sort(H(i,:)),1:13), 1:40)));
H = cruce_blx(pob_ackley(40, 3), 0.5, [-10 10]);
fprintf('BLX: hijos dentro de [-10, 10] = %d\n', all(abs(H(:)) <= 10));
H = cruce_aritmetico(pob_inventario(40));
fprintf('Aritmetico: max |sum(Q/2+S) - I| = %.2e\n', max(abs(sum(H(:,1:k)/2,2) + sum(H(:,k+1:2*k),2) - p.I)));

fprintf('--- Mutacion ---\n');
H = mut_viajero(pob_viajero(40), 1);
fprintf('Swap: siguen siendo permutaciones = %d\n', all(arrayfun(@(i) isequal(sort(H(i,:)),1:13), 1:40)));
H0 = pob_ackley(1000, 3); H = mut_ackley(H0, 0.3, 1, [-10 10]);
fprintf('Ackley: fraccion mutada = %.3f (Pm = 0.3)\n', mean(any(H ~= H0, 2)));
H0 = pob_inventario(1000); H = mut_inventario(H0, 0.3, 0.1);
fprintf('Inventario: fraccion mutada = %.3f, max |restriccion| = %.2e\n', mean(any(H ~= H0, 2)), ...
   max(abs(sum(H(:,1:k)/2,2) + sum(H(:,k+1:2*k),2) - p.I)));

fprintf('--- Corrida completa (100 iteraciones, torneo) ---\n');
op = struct('n', 60, 'iter', 100, 'seleccion', @sel_torneo, 'Pm', 0.2, 'sigma', 0.5);
r = algoritmo_genetico(definir_problema('viajero'), op);
fprintf('Viajero: %d min en %.2f s, ruta %s\n', r.mejor_f, r.tiempo, mat2str(r.mejor_x));
r = algoritmo_genetico(definir_problema('ackley', 3), op);
fprintf('Ackley d=3: %.6f en %.2f s\n', r.mejor_f, r.tiempo);
op.sigma = 0.05;
r = algoritmo_genetico(definir_problema('inventario'), op);
fprintf('Inventario: Z = %.4f en %.2f s\n', r.mejor_f, r.tiempo);
disp(reshape(r.mejor_x, 5, 3)');
```

generar_salidas.m

```
function generar_salidas()
% GENERAR_SALIDAS Crea las tablas (LaTeX) y figuras del informe a partir de
% los resultados guardados por experimentos.m.
res = fullfile('..', 'resultados');
tab = fullfile('..', 'informe', 'tablas');
fig = fullfile('..', 'informe', 'figuras');
if ~exist(tab, 'dir'), mkdir(tab); end
```

```
if ~exist(fig, 'dir'), mkdir(fig); end
load(fullfile(res, 'E1.mat'), 'E1', 'nombres_sel', 'optimos');
load(fullfile(res, 'E2.mat'), 'E2', 'iteraciones');
load(fullfile(res, 'E3.mat'), 'E3');
load(fullfile(res, 'E4.mat'), 'E4', 'dimensiones', 'decision');

% ---------------- Estudio 1: seleccion ----------------
tabla_viajero(fullfile(tab, 't1_viajero.tex'), E1.viajero, nombres_sel, 'M\''etodo', ...
   'Estudio 1 -- problema del viajero: m\''etodo de selecci\''on (100 iteraciones).', 'tab:t1v');
tabla_ackley(fullfile(tab, 't1_ackley.tex'), E1.ackley, nombres_sel, 'M\''etodo', ...
   'Estudio 1 -- funci\''on Ackley ($d=3$): m\''etodo de selecci\''on (100 iteraciones).', 'tab:t1a');
tabla_inventario(fullfile(tab, 't1_inventario.tex'), E1.inventario, nombres_sel, 'M\''etodo', ...
   'Estudio 1 -- inventarios: m\''etodo de selecci\''on (100 iteraciones).', 'tab:t1i');

% ---------------- Estudio 2: iteraciones ----------------
et = arrayfun(@num2str, iteraciones, 'UniformOutput', false);
tabla_viajero(fullfile(tab, 't2_viajero.tex'), E2.viajero, et, 'Iter.', ...
   'Estudio 2 -- problema del viajero: n\''umero de iteraciones (selecci\''on por elitismo).', 'tab:t2v');
tabla_ackley(fullfile(tab, 't2_ackley.tex'), E2.ackley, et, 'Iter.', ...
   'Estudio 2 -- funci\''on Ackley ($d=3$): n\''umero de iteraciones (selecci\''on por elitismo).', 'tab:t2a');
tabla_inventario(fullfile(tab, 't2_inventario.tex'), E2.inventario, et, 'Iter.', ...
   'Estudio 2 -- inventarios: n\''umero de iteraciones (selecci\''on por elitismo).', 'tab:t2i');

% ---------------- Estudio 3: mutacion ----------------
etv = arrayfun(@(r) sprintf('%.2f', r.op.Pm), E3.viajero, 'UniformOutput', false);
tabla_viajero(fullfile(tab, 't3_viajero.tex'), E3.viajero, etv, '$P_m$', ...
   'Estudio 3 -- problema del viajero: probabilidad de mutaci\''on $P_m$ (elitismo, 50 iteraciones).', 'tab:t3v', false);
eta = arrayfun(@(r) sprintf('%.2f / %.1f', r.op.Pm, r.op.sigma), E3.ackley, 'UniformOutput', false);
tabla_ackley(fullfile(tab, 't3_ackley.tex'), E3.ackley, eta, '$P_m$ / $\sigma_m$', ...
   'Estudio 3 -- funci\''on Ackley ($d=3$): probabilidad $P_m$ y magnitud $\sigma_m$ de la mutaci\''on (elitismo, 50
       iteraciones).', 'tab:t3a', false);
eti = arrayfun(@(r) sprintf('%.2f / %.2f', r.op.Pm, r.op.sigma), E3.inventario, 'UniformOutput', false);
tabla_inventario(fullfile(tab, 't3_inventario.tex'), E3.inventario, eti, '$P_m$ / $\sigma_r$', ...
   'Estudio 3 -- inventarios: probabilidad $P_m$ y magnitud relativa $\sigma_r$ de la mutaci\''on (elitismo, 200
       iteraciones).', 'tab:t3i', false);

% ---------------- Estudio 4: dimensiones ----------------
fid = fopen(fullfile(tab, 't4_ackley.tex'), 'w');
fprintf(fid, '%% Generado automaticamente por generar_salidas.m\n');
fprintf(fid, '\\begin{table}[!t]\n\\centering\n\\scriptsize\n\\setlength{\\tabcolsep}{3pt}\n');
fprintf(fid, '\\caption{Estudio 4 -- funci\\''on Ackley: n\\''umero de dimensiones (elitismo, %d iteraciones, $P_m=%.2f$,
    $\\sigma_m=%.1f$).}\n', ...
   decision.ackley.iter, decision.ackley.Pm, decision.ackley.sigma);
fprintf(fid, '\\label{tab:t4a}\n\\begin{tabular}{rrrrrrr}\n\\hline\n');
fprintf(fid, '$d$ & $t$ (ms) & $f(\\hat{X})$ & Prom. & Desv. & RMSE & $\\|\\hat{X}\\|_\\infty$ \\\\\n\\hline\n');
for k = 1:numel(E4)
   r = E4(k);
   fprintf(fid, '%d & %.2f & %.4f & %.4f & %s & %s & %s \\\\\n', dimensiones(k), 1000*r.tiempo, ...
      r.mejor_f, r.promedio, sci(r.desv), sci(r.rmse), sci(max(abs(r.mejor_x))));
end
fprintf(fid, '\\hline\n\\end{tabular}\n\\end{table}\n');
fclose(fid);
% Individuos completos del estudio 4 (anexo)
fid = fopen(fullfile(res, 'estudio4_mejores_individuos.csv'), 'w');
for k = 1:numel(E4)
   fprintf(fid, 'd=%d,%s\n', dimensiones(k), strjoin(arrayfun(@(v) sprintf('%.6g', v), E4(k).mejor_x, 'UniformOutput',
       false), ','));
end
fclose(fid);

% ---------------- Figuras ----------------
nombres_prob = {'viajero', 'ackley', 'inventario'};
titulos = {'Problema del viajero', 'Funci\''on Ackley ($d=3$)', 'Inventarios'};
ylab = {'Tiempo (min)', '$f(\hat{X})$', '$Z$'};
ylab_conv = {'Tiempo (min)', '$f(\hat{X}) - (3-e)$', '$Z$'};
f = figure('Visible', 'off', 'Theme', 'light', 'Units', 'centimeters', 'Position', [0 0 8.8 14]);
tl = tiledlayout(3, 1, 'TileSpacing', 'compact', 'Padding', 'compact');
for ip = 1:3
   nexttile; hold on; box on; grid on;
   for is = 1:3
      c = E1.(nombres_prob{ip})(is).conv;
      if ip == 2, c = c - (3 - exp(1)); end % error respecto al optimo
      plot(0:size(c,1)-1, mean(c, 2), 'LineWidth', 1.2);
   end
   if ip == 2, set(gca, 'YScale', 'log'); end
   title(titulos{ip}, 'Interpreter', 'latex', 'FontSize', 9);
   ylabel(ylab_conv{ip}, 'Interpreter', 'latex');
   if ip == 1, legend(nombres_sel, 'Location', 'northeast', 'FontSize', 7); end
end
xlabel(tl, 'Generaci\''on', 'Interpreter', 'latex');
exportgraphics(f, fullfile(fig, 'convergencia_seleccion.pdf'), 'ContentType', 'vector');
exportgraphics(f, fullfile(res, 'convergencia_seleccion.png'), 'Resolution', 150);
close(f);
```

```
87
88  f = figure('Visible', 'off', 'Theme', 'light', 'Units', 'centimeters', 'Position', [0 0 8.8 12]);
89  tl = tiledlayout(3, 1, 'TileSpacing', 'compact', 'Padding', 'compact');
90  for ip = 1:3
91     R = E3.(nombres_prob{ip})(1:7);
92     nexttile; box on; grid on; hold on;
93     errorbar([0 0.05 0.1 0.2 0.4 0.7 1.0], [R.promedio], [R.desv], 'o-', 'LineWidth', 1.1, 'MarkerSize', 4);
94     title(titulos{ip}, 'Interpreter', 'latex', 'FontSize', 9);
95     ylabel(ylab{ip}, 'Interpreter', 'latex');
96  end
97  xlabel(tl, 'Probabilidad de mutaci\''on $P_m$', 'Interpreter', 'latex');
98  exportgraphics(f, fullfile(fig, 'efecto_pm.pdf'), 'ContentType', 'vector');
99  exportgraphics(f, fullfile(res, 'efecto_pm.png'), 'Resolution', 150);
100 close(f);
101
102 f = figure('Visible', 'off', 'Theme', 'light', 'Units', 'centimeters', 'Position', [0 0 8.8 6]);
103 errorbar(dimensiones, [E4.promedio], [E4.desv], 'o-', 'LineWidth', 1.2, 'MarkerSize', 4); hold on;
104 yline(3 - exp(1), '--', '$3-e$', 'Interpreter', 'latex');
105 set(gca, 'XScale', 'log'); grid on; box on;
106 xlabel('N\''umero de dimensiones $d$', 'Interpreter', 'latex');
107 ylabel('$f(\hat{X})$ promedio', 'Interpreter', 'latex');
108 exportgraphics(f, fullfile(fig, 'dimensiones.pdf'), 'ContentType', 'vector');
109 exportgraphics(f, fullfile(res, 'dimensiones.png'), 'Resolution', 150);
110 close(f);
111 fprintf('Tablas y figuras generadas.\n');
112 end
113
114 % ========================================================================
115 function tabla_viajero(archivo, R, etiquetas, col, caption, label, con_tiempo)
116 if nargin < 7, con_tiempo = true; end
117 fid = fopen(archivo, 'w');
118 fprintf(fid, '%% Generado automaticamente por generar_salidas.m\n');
119 fprintf(fid, '\\begin{table}[!t]\n\\centering\n\\scriptsize\n\\setlength{\\tabcolsep}{2.5pt}\n');
120 fprintf(fid, '\\caption{%s}\n\\label{%s}\n', caption, label);
121 if con_tiempo
122    fprintf(fid, '\\begin{tabular}{lrrrrrl}\n\\hline\n');
123    fprintf(fid, '%s & $t$ (ms) & $f(\\hat{X})$ & Prom. & Desv. & RMSE$^*$ & $\\hat{X}$ (ruta) \\\\\n\\hline\n', col);
124 else
125    fprintf(fid, '\\begin{tabular}{lrrrrl}\n\\hline\n');
126    fprintf(fid, '%s & $f(\\hat{X})$ & Prom. & Desv. & RMSE$^*$ & $\\hat{X}$ (ruta) \\\\\n\\hline\n', col);
127 end
128 for k = 1:numel(R)
129    ruta = strjoin(arrayfun(@num2str, R(k).mejor_x, 'UniformOutput', false), '-');
130    if con_tiempo
131       fprintf(fid, '%s & %.1f & %d & %.1f & %.2f & %.2f & %s \\\\\n', etiquetas{k}, ...
132          1000*R(k).tiempo, R(k).mejor_f, R(k).promedio, R(k).desv, R(k).rmse, ruta);
133    else
134       fprintf(fid, '%s & %d & %.1f & %.2f & %.2f & %s \\\\\n', etiquetas{k}, ...
135          R(k).mejor_f, R(k).promedio, R(k).desv, R(k).rmse, ruta);
136    end
137 end
138 fprintf(fid, '\\hline\n\\multicolumn{%d}{l}{$^*$RMSE respecto al \\''optimo exacto (509 min, Held-Karp).}\n', 6 +
      con_tiempo);
139 fprintf(fid, '\\end{tabular}\n\\end{table}\n');
140 fclose(fid);
141 end
142
143 function tabla_ackley(archivo, R, etiquetas, col, caption, label, con_tiempo)
144 if nargin < 7, con_tiempo = true; end
145 fid = fopen(archivo, 'w');
146 fprintf(fid, '%% Generado automaticamente por generar_salidas.m\n');
147 fprintf(fid, '\\begin{table*}[!t]\n\\centering\n\\scriptsize\n\\setlength{\\tabcolsep}{2.5pt}\n');
148 fprintf(fid, '\\caption{%s}\n\\label{%s}\n', caption, label);
149 if con_tiempo
150    fprintf(fid, '\\begin{tabular}{lrrrrrl}\n\\hline\n');
151    fprintf(fid, '%s & $t$ (ms) & $f(\\hat{X})$ & Prom. & Desv. & RMSE & $\\hat{X}$ \\\\\n\\hline\n', col);
152 else
153    fprintf(fid, '\\begin{tabular}{lrrrrl}\n\\hline\n');
154    fprintf(fid, '%s & $f(\\hat{X})$ & Prom. & Desv. & RMSE & $\\hat{X}$ \\\\\n\\hline\n', col);
155 end
156 for k = 1:numel(R)
157    x = ['(' strjoin(arrayfun(@sci, R(k).mejor_x, 'UniformOutput', false), ', ') ')'];
158    if con_tiempo
159       fprintf(fid, '%s & %.2f & %.6f & %.6f & %s & %s & %s \\\\\n', etiquetas{k}, 1000*R(k).tiempo, ...
160          R(k).mejor_f, R(k).promedio, sci(R(k).desv), sci(R(k).rmse), x);
161    else
162       fprintf(fid, '%s & %.6f & %.6f & %s & %s & %s \\\\\n', etiquetas{k}, ...
163          R(k).mejor_f, R(k).promedio, sci(R(k).desv), sci(R(k).rmse), x);
164    end
165 end
166 fprintf(fid, '\\hline\n\\end{tabular}\n\\end{table*}\n');
167 fclose(fid);
168 end
```

```
function tabla_inventario(archivo, R, etiquetas, col, caption, label, con_tiempo)
if nargin < 7, con_tiempo = true; end
fid = fopen(archivo, 'w');
fprintf(fid, '%% Generado automaticamente por generar_salidas.m\n');
fprintf(fid, '\\begin{table*}[!t]\n\\centering\n\\scriptsize\n\\setlength{\\tabcolsep}{3pt}\n');
fprintf(fid, '\\caption{%s Valores monetarios en miles de COP.}\n\\label{%s}\n', caption, label);
if con_tiempo
   fprintf(fid, '\\begin{tabular}{lrrrrlll}\n\\hline\n');
   fprintf(fid, '%s & $t$ (s) & $f(\\hat{X})$ & Prom. & Desv. & $\\hat{Q}$ & $\\hat{S}$ & $\\hat{K}$ \\\\\n\\hline\n', col
       );
else
   fprintf(fid, '\\begin{tabular}{lrrrlll}\n\\hline\n');
   fprintf(fid, '%s & $f(\\hat{X})$ & Prom. & Desv. & $\\hat{Q}$ & $\\hat{S}$ & $\\hat{K}$ \\\\\n\\hline\n', col);
end
for k = 1:numel(R)
   x = R(k).mejor_x;
   v = @(a) ['(' strjoin(arrayfun(@(y) sprintf('%.0f', y), a, 'UniformOutput', false), ', ') ')'];
   if con_tiempo
      fprintf(fid, '%s & %.2f & %.2f & %.2f & %.2f & %s & %s & %s \\\\\n', etiquetas{k}, R(k).tiempo, ...
         R(k).mejor_f, R(k).promedio, R(k).desv, v(x(1:5)), v(x(6:10)), v(x(11:15)));
   else
      fprintf(fid, '%s & %.2f & %.2f & %.2f & %s & %s & %s \\\\\n', etiquetas{k}, ...
         R(k).mejor_f, R(k).promedio, R(k).desv, v(x(1:5)), v(x(6:10)), v(x(11:15)));
   end
end
fprintf(fid, '\\hline\n\\end{tabular}\n\\end{table*}\n');
fclose(fid);
end

function s = sci(x)
% Numero en notacion cientifica compacta para LaTeX
if x == 0
   s = '0';
elseif abs(x) >= 1e-2 && abs(x) < 1e3
   s = sprintf('%.3f', x);
else
   e = floor(log10(abs(x)));
   s = sprintf('$%.2f{\\times}10^{%d}$', x/10^e, e);
end
end
```